

Introduction

CATIA V5

CATIA V5 is a powerful, comprehensive software suite for Computer-Aided Design (CAD), Computer-Aided Manufacturing (CAM), and Computer-Aided Engineering (CAE) developed by Dassault Systèmes. It is an industry standard used in sectors like automotive and aerospace to create detailed 3D models and 2D drawings for product development, from initial design to final manufacturing. The software is organized into "workbenches" that focus on specific tasks, such as Part Design, Assembly Design, and Drafting.

Key features and functions

- **3D and 2D Design:**
CATIA V5 combines powerful 3D parametric modeling with 2D drafting tools, allowing users to create intricate designs from simple sketches to complex assemblies.
- **Industry Standard:**
It is widely used across many industries, including automotive, aerospace, and consumer goods.
- **Parametric Modeling:**
It uses 3D parametric features, meaning designs can be built from 2D sketches and then modified by changing the parameters.
- **Organized Workbenches:**
The software is structured into different workbenches, which are specialized environments for different tasks, like the Sketcher workbench for drawing or the Assembly Design workbench for combining parts.
- **Comprehensive Process:**
It supports the entire product lifecycle, from initial design and engineering analysis to the final manufacturing process.
- **Visualization:**
It allows designers to create a realistic on-screen representation of a product without building a physical prototype, making it easier to visualize and modify ideas.
- **Analysis:**
It includes tools for performing complex simulations, such as static and dynamic analysis, heat transfer, and fluid flow.
- **Efficiency:**
It provides tools for generating and modifying designs quickly and accurately, speeding up the design process.

3D printing

3D printing, or additive manufacturing, is a process that builds a three-dimensional object layer by layer from a digital model. It works by adding material, such as plastics, metals, or resins, under computer control to construct an object from a digital file, which is the opposite of traditional subtractive manufacturing where material is removed from

a larger block. This allows for the creation of complex shapes with less material waste and enables rapid prototyping across various industries.

How it works

1. Create a 3D model:

A digital 3D model is created using CAD (Computer-Aided Design) software or obtained from a 3D scan.

2. Slice the model:

The 3D model is then processed by "slicing" software, which breaks it down into hundreds or thousands of thin, horizontal layers.

3. Print the layers:

The 3D printer reads the instructions for each layer and builds the object by adding, fusing, or solidifying material, layer by layer, until the final object is complete.

4. Post-processing:

After printing, the object may require additional steps like removing support structures, sanding, or painting to finish it.

Key aspects

- Additive vs. subtractive:

3D printing is an "additive" process, building up an object, while traditional methods are "subtractive," carving away material. This means 3D printing typically generates less material waste.

- Materials:

A wide range of materials can be used, including plastics (like PLA or ABS), metals, and resins, depending on the specific printing technology.

- Technologies:

Common 3D printing methods include Fused Deposition Modeling (FDM), Stereolithography (SLA), and Selective Laser Sintering (SLS).

- Applications:

3D printing is used in many fields, such as engineering, healthcare, education, and manufacturing for creating prototypes and complex parts.

Igus robots

igus robots are a range of modular, low-cost automation solutions primarily made from wear-resistant plastics, designed to offer customizable and affordable robotics for various industries. The company's offerings include articulated arms, cobots, delta robots, linear robots, and the humanoid robot Iggy Rob, all controllable via the user-friendly igus Robot Control software. Their focus is on making automation accessible to a wider range of businesses, particularly small and medium-sized companies.

Key features and components

- Modular design: igus's modular system allows users to combine different components to create a customized robot for specific tasks, avoiding expensive new developments.
- Low-cost automation: The use of plastic components and a focus on affordability makes automation solutions more accessible, enabling a quick return on investment.
- Variety of robot types: The product range includes a diverse selection of robots, such as articulated arms, cobots, delta robots, linear robots, and the humanoid Iggy Rob.

- [igus Robot Control](#): This software is designed for easy, intuitive programming and control of the robots, supporting various communication protocols like Ethernet and Wi-Fi.
- **Wear-resistant materials**: igus specializes in using durable, low-friction plastics for its robotic components, which are designed for reliability and require minimal lubrication.

Arduino based Automation

Arduino projects involve using an open-source electronic platform, consisting of a physical board and software, to create interactive projects like those for robotics, home automation, and art installations. These projects are built by connecting the board to components like sensors, motors, and LEDs, and then writing code in the Arduino IDE to tell the board how to control them. The code is written in a C/C++ based language, but many resources and tutorials are available to help beginners even without prior experience.

Arduino projects work

- **Hardware**: The physical Arduino board is a microcontroller that acts as a small computer. It has digital and analog pins to connect with external components, such as sensors that read information from the environment and actuators like motors that perform actions.
- **Software**: The Arduino IDE (Integrated Development Environment) is where you write the code, often referred to as a "sketch". This code contains a `setup()` function that runs once and a `loop()` function that repeats indefinitely.
- **Process**: You write a program in the IDE that tells the board what to do. For example, you might write code to read a sensor, then use that information to turn on a light or control a motor.
- **Uploading**: After writing the code, you upload it to the Arduino board via a USB cable. Once uploaded, the board executes the program automatically.

Experiment 1: Install and Explore CPRog Software Features and Safety Protocols

Aim

To install CPRog software on a Windows computer, explore its major features for robot programming, and understand the safety protocols associated with its operation in laboratory and industrial environments.

Apparatus/Requirements

1. Hardware:
 - A Windows PC or laptop (Windows 10/11, 64-bit recommended)
 - USB cable or CAN/Ethernet adapter (depending on hardware used with CPRog)
 - Target hardware device (robot controller / microcontroller board if testing connection)
 2. Software:
 - CPRog installer (from vendor/official website)
 - Supporting drivers (e.g., FTDI drivers, PCAN drivers, or equivalent)
 - Windows administrator account for installation
 3. Accessories:
 - Internet connection (for downloading drivers/updates)
 - External storage (optional, for software backup)
-

Theory

1. Introduction to CPRog Software

CPRog is a robotic programming software developed by Commonplace Robotics in collaboration with Igus GmbH. It provides an intuitive environment for programming Igus Robolink robotic arms and similar systems. The software is designed for ease of use, especially in academic and training setups, while also being powerful enough for industrial prototyping.

The key advantage of CPRog is its graphical user interface (GUI) that allows programming robots without deep coding skills. Tasks like pick-and-place, path following, and material handling can be set up quickly using block commands.

2. Major Features of CPRog

- **Jogging Mode:** Enables manual control of each robot joint or Cartesian movement using sliders.
 - **Program Creation:** Supports block-based task creation or text-based commands.
 - **Simulation Environment:** Allows offline testing of programs before execution.
 - **Coordinate Systems:** Supports world, tool, and joint coordinate frames.
 - **Tool Management:** Lets users define grippers, vacuum systems, or other end-effectors.
 - **Import/Export Options:** Programs can be saved, modified, and reused.
 - **Integration with Safety Hardware:** Supports emergency stop and collision avoidance protocols.
-

3. Safety Protocols in Robotic Programming

Safety is a critical part of working with robots. Even a small educational robotic arm can move unexpectedly and cause injury or damage. CPRog emphasizes safety through software limits and hardware integration. Key safety protocols include:

1. **Emergency Stop** – A physical red switch must be present to cut power instantly.
 2. **Joint Limits** – Software prevents arms from moving beyond safe angles.
 3. **Collision Avoidance** – Virtual fences/zones can be defined in CPRog.
 4. **Simulation First** – All programs should be tested virtually before actual execution.
 5. **Safe Distance** – Users should avoid entering the operating envelope of the robot when it is powered.
-

4. Applications of CPRog

- Educational training in robotics and automation.
 - Pick-and-place task demonstrations.
 - Research on robotic motion planning.
 - Teaching coordinate systems (DH parameters, joint space, Cartesian space).
-

Procedure

1. Preparation

- Ensure that the computer meets the system requirements.
- Obtain the CPRog installer from Igus official site or lab-provided media.
- Verify that antivirus/firewall settings allow the installer to run.

2. Installation

- Run the setup file (e.g., CPRog_setup.exe).
- Accept the license agreement and choose installation path.
- Complete installation and allow creation of desktop shortcut.

3. Launching Software

- Double-click the CPRog icon on the desktop.
- Observe the splash screen and wait for the main interface.

4. Connecting to Robot Controller

- Connect the computer to the Igus controller via USB or Ethernet.
- In CPRog, open **Connection Settings** → select communication port.
- Verify successful connection through status indicator (green light).

5. Exploring Features

- Open **Jog Mode** and manually move the robot joints.
- Switch to **Cartesian Control** to test movement along X, Y, Z axes.
- Save a few positions as “Points.”
- Create a simple program: Move from Point A → Point B → return to Home.
- Run the program first in simulation mode.

6. Safety Protocol Verification

- Locate the Emergency Stop switch and ensure it is functional.
- Check joint limits by attempting to move beyond safe values (software should block motion).
- Test collision zones if configured.

7. Documentation

- Record version number of CPRog installed.
- Take screenshots of interface, jog mode, and program editor.
- Note observations about safety protocols.

Observations

S. No	Task Performed	Observation/Result
1	Installation completed	Successful
2	Connection with robot	Connected/Not Connected
3	Jog mode tested	Working / Not Working
4	Emergency stop test	Functional / Not Functional
5	Program execution (simulation)	Successful

screenshots attached:

- Screenshot of Jog Mode
- Screenshot of Program Editor
- Screenshot of Simulation Window

Precautions

1. Ensure that the robot is powered OFF during installation.
2. Use only the official version of CPRog to avoid malware risks.
3. Keep the working area clear of obstacles before testing.
4. Do not bypass the emergency stop circuit.
5. Always test the program in **simulation mode first** before execution.
6. Do not attempt to manually push/move the robot while powered ON.

Result

The CPRog software was successfully installed on a Windows computer. The main features such as jogging, program creation, and simulation were explored. Safety protocols such as joint limit checks, collision avoidance, and emergency stop functionality were studied.

Experiment 2: Program an IGUS Robot for Pick and Place Tasks with a Magnetic Gripper

Aim

To program an IGUS robotic arm using CPRog software to perform pick-and-place operations with the help of a magnetic gripper, and to study its applications in material handling of ferromagnetic objects.

Apparatus/Software Required

1. IGUS Robolink Robotic Arm (4–6 DOF).
 2. Magnetic Gripper Module (electromagnetic).
 3. Robot Controller with CPRog software installed on Windows PC.
 4. Power supply unit.
 5. Metallic workpieces (nuts, bolts, washers).
 6. Connection cable (USB/Ethernet).
 7. Emergency stop switch.
-

Theory

1. Pick and Place Operation

Pick-and-place is one of the most fundamental robotic operations where the robot:

- Moves to the object location (pick point).
- Activates a gripping mechanism.
- Transfers the object along a defined path.
- Places the object at a new location (place point).

This principle is widely used in manufacturing, assembly lines, logistics, and automation labs.

2. Magnetic Gripper Working Principle

- A magnetic gripper uses an electromagnet to attract ferromagnetic materials (like iron, steel).

- When current flows through the coil, a magnetic field is generated.
- This field attracts metallic objects, holding them securely.
- When the current is switched off, the object is released.

Advantages:

- Simple design, no moving mechanical parts.
- Quick switching (ON/OFF).
- Suitable for flat, smooth metallic objects.

Limitations:

- Works only with ferromagnetic materials.
 - Holding force decreases with rough/uneven surfaces.
 - May attract unwanted metallic debris.
-

3. Applications of Magnetic Grippers

- Handling of metallic sheets in automobile industries.
 - Pick-and-place of nuts, bolts, and machine parts.
 - Automated feeding systems in CNC workshops.
 - Robotics education and training.
-

Procedure

1. Setup
 - Mount the magnetic gripper onto the IGUS robot flange.
 - Connect the gripper to the robot controller (electrical output port).
 - Place metallic objects at a defined Pick Area.
2. Software Preparation
 - Launch CPRog software.
 - Connect the computer to the robot controller.
 - Verify the robot connection (green status indicator).
3. Programming Steps
 - a. Open Jog Mode and move the robot above the pick-up location.
 - b. Save this as Approach Point (P0).

- c. Lower the arm to touch the metallic object → Save as Pick Point (P1).
- d. Insert command: Magnet ON (to energize gripper).
- e. Raise the robot slightly upward to avoid collisions.
- f. Move the arm to the desired destination → Save as Place Point (P2).
- g. Insert command: Magnet OFF (to release object).
- h. Return the robot to Home Position (P_H).
- i. Save and name the program as *PickPlace_Magnet.prg*.

4. Simulation and Execution

- Run the program in Simulation Mode to check trajectory.
- If no errors, execute on the actual robot.
- Observe if the metallic object is successfully transferred.

5. Repeat

- Modify the program for multiple objects.
- Test speed variations and record results.

Observations

S. No	Task	Observation (✓/X)
1	Magnetic gripper mounted	
2	Connection with controller	
3	Magnet ON/OFF tested	
4	Pick and Place cycle executed	
5	Object weight handled	

Attached screenshots :

- Program flow in CPRog.
 - Robot picking up metallic objects.
-

Precautions

1. Ensure only ferromagnetic objects are used.

2. Do not exceed the rated payload of the gripper.
 3. Keep hands away from the object during magnetic pick-up.
 4. Avoid working near sensitive electronic devices (magnet interference).
 5. Always test in simulation before execution.
-

Result

The IGUS robotic arm was successfully programmed using CPRog to perform pick-and-place tasks with a magnetic gripper. The experiment demonstrated electromagnetic gripping for robotic material handling,

Experiment 3: Program an IGUS Robot for Pick and Place Tasks with a Vacuum Gripper

Aim

To program an IGUS robotic arm using CPRog software to perform pick-and-place operations with the help of a vacuum gripper, and to understand its use in handling lightweight non-metallic objects.

Apparatus/Software Required

1. IGUS Robolink Robotic Arm (4–6 DOF).
 2. Vacuum Gripper Module with suction cups.
 3. Vacuum generator or pump with tubing.
 4. Robot Controller with CPRog software.
 5. Objects for testing: lightweight, flat, non-porous (plastic blocks, acrylic sheets).
 6. Emergency stop switch.
 7. Power supply and connectors.
-

Theory

1. Vacuum Gripper Working Principle

- A vacuum gripper creates suction pressure using a vacuum pump or venturi system.
- The suction cup attaches firmly to the object's surface.
- When vacuum is released, the object detaches.

Advantages:

- Can handle a wide variety of materials (plastics, glass, cardboard).
- Gentle handling (no scratches).
- Simple ON/OFF control.

Limitations:

- Works best with flat, non-porous surfaces.
- Not suitable for rough, irregular, or porous items.
- Suction force depends on vacuum pressure and surface area.

2. Applications of Vacuum Grippers

- Packaging industries (picking cartons, bottles, boxes).
 - Semiconductor handling (chips, wafers).
 - Food industries (sealed packets, containers).
 - Lightweight assembly lines.
-

Procedure

1. Setup

- Attach the vacuum gripper to the IGUS robot's end flange.
- Connect suction tubing to the vacuum generator/pump.
- Place test objects (plastic/acrylic) at the Pick Area.

2. Software Preparation

- Launch CPRog software on the computer.
- Connect robot controller to the computer.
- Verify the robot status is "Ready."

3. Programming Steps

- a. Jog the robot above the object → Save as Approach Point (P0).
- b. Lower to object surface → Save as Pick Point (P1).
- c. Insert command: Vacuum ON (to activate suction).
- d. Lift the robot slightly upward.
- e. Move to the destination → Save as Place Point (P2).
- f. Insert command: Vacuum OFF (release object).
- g. Move back to Home Position (P_H).
- h. Save program as *PickPlace_Vacuum.prg*.

4. Simulation and Execution

- Run the program in simulation first.
- Execute on actual robot and observe suction holding.
- Adjust speed to avoid dropping the object.

5. Repetition

- Test different lightweight objects.

- Record whether suction is successful or not.

Observations

Object Tested	Surface Type	Vacuum Grip Success (✓/X)	Remarks
Plastic Block	Smooth		
Acrylic Sheet	Flat		
Cardboard	Porous		

Attached photos/screenshots :

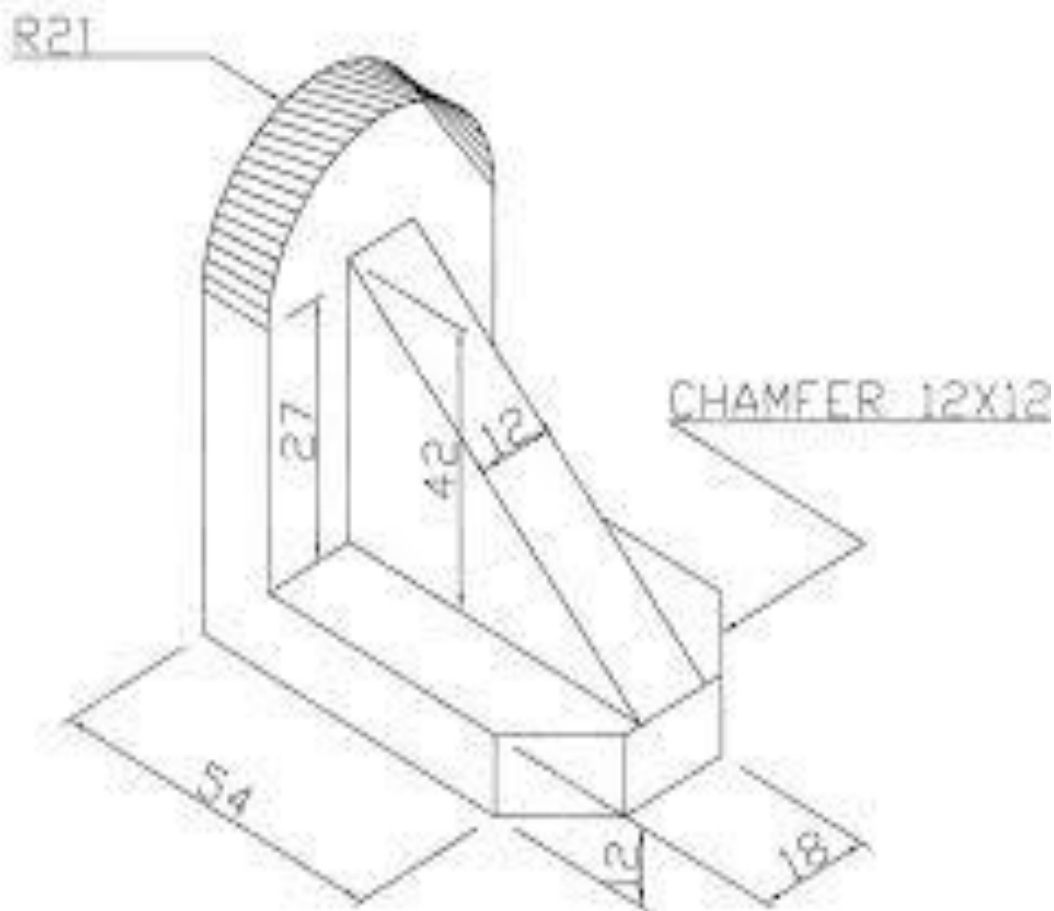
- Suction holding an object.
- Program editor with Vacuum ON/OFF commands.

Precautions

1. Use only lightweight, non-porous objects.
2. Ensure vacuum tubing is airtight (no leaks).
3. Do not exceed the suction capacity of the gripper.
4. Avoid sudden or jerky robot movements while carrying an object.
5. Always keep the emergency stop accessible.

Result

The IGUS robotic arm was successfully programmed in CPRog to perform pick-and-place operations using a vacuum gripper. The experiment demonstrated suction-based handling for lightweight, smooth-surfaced objects.



Experiment 4: Create a simple geometry using CAD software

Aim: To create a simple geometry using CAD software

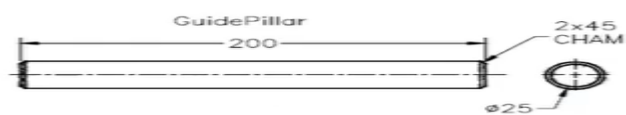
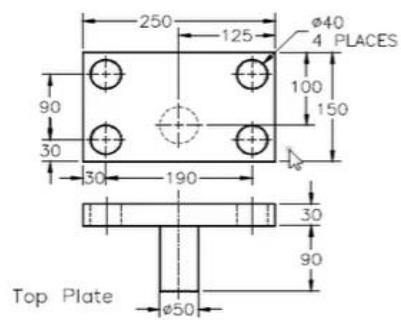
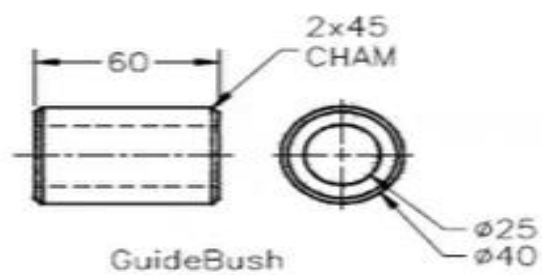
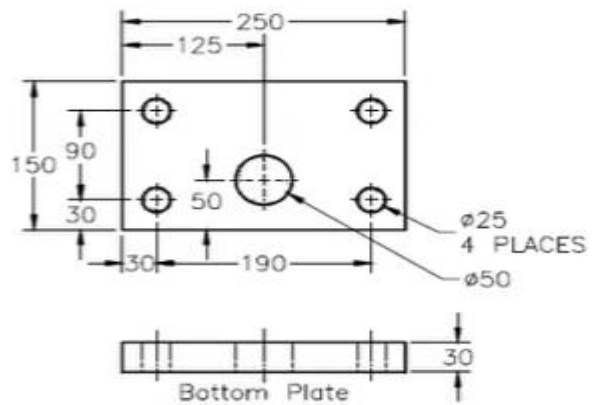
Software required: CATIA V5 R21

Commands required: line, rectangle, pad, constraint coincidence, chamfer, tangent fillet.

Procedure:

1. Using Sketcher, construct a rectangle(horizontal) of dimensions 54mm×42mm
1. Exit work bench and perform padding for 12mm
2. Construct another rectangle(Vertical) of dimensions 48mm×42mm
3. Using constraint coincidence command align these two rectangles.
4. Perform padding of 12mm for vertical rectangle.
5. Using chamfer tool perform chamfering on the bottom edge of horizontal rectangle for 12mm×12mm.
6. Using tangent fillet select the edges and surface of vertical rectangle and perform a fillet of R21.
7. Using sketcher draw a right angled triangle on the bottom surface of the horizontal rectangle and perform padding of thickness 12mm to obtain the rib plate at the center

Result: Hence, a simple 2D geometry was modelled using CATIA software



Experiment 5: Design and assemble multiple CAD components with proper fit.

Aim: To design and assemble multiple CAD components with proper fit.

Software required: CATIA V5 R21

Commands required: Sketcher module, part module, existing component, product tree, move, rotate, snap

Procedure:

1. Model the bottom plate using the given dimensions using Sketcher and part modelling
2. Model the top plate using the given dimensions
 1. Model the guide pillar plate using the given dimensions
 2. Model guide bush using the given dimensions
 3. Save all the components and save them in a folder of a defined path
 4. Open Catia -Mechanical Design-assembly module.
 5. Upload each component from the saved folder using the existing component and the existing component with positioning.
 6. Position the bottom plate at the bottom.
 7. Using the snap tool, insert guide pillars into the four holes of the bottom plate
 8. Similarly, insert four guide bushes into the top surface of the guide pillars
 9. Insert the top plate into the guide bushes using move, rotate and snap commands.

Result: Hence, multiple CAD components were designed and assembled with proper fit.

Experiment 6: Develop a 3D parametric model by adjusting key parameters.

Aim: To develop a 3D parametric model by adjusting key parameters.

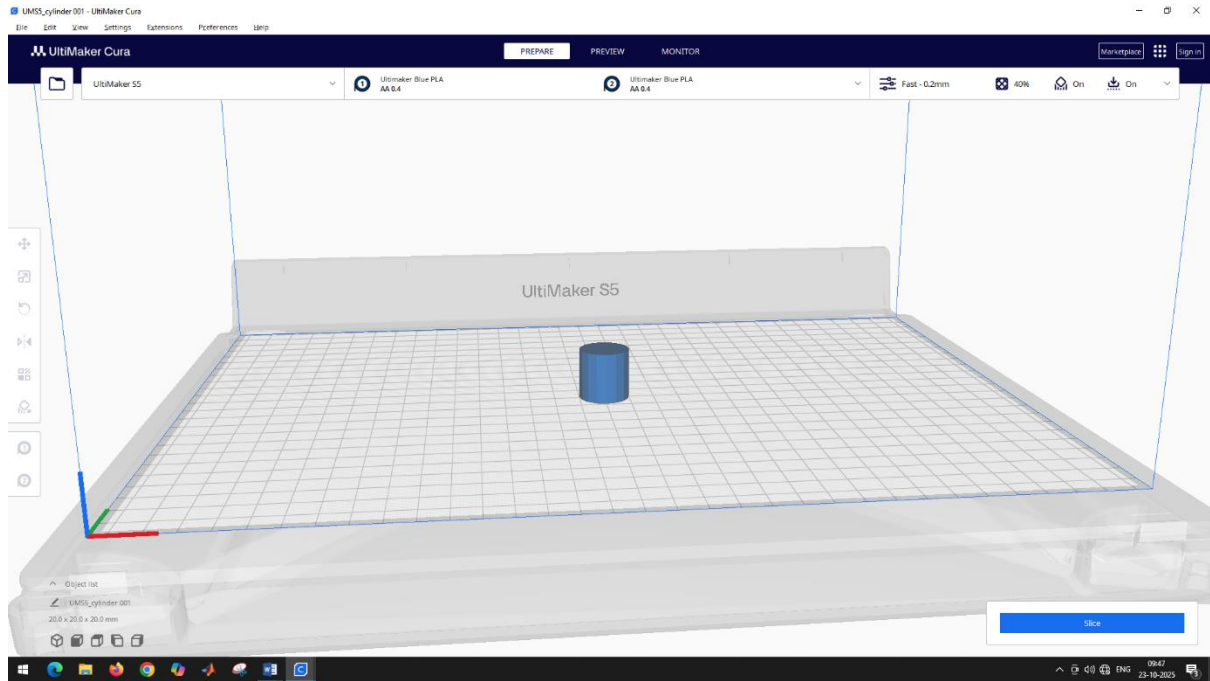
Software required: CATIA V5 R21

Commands required: sketcher module, part module, formula catalogue, formula editor

Procedure:

1. Draw a rectangle using sketcher module
2. Perform padding using part module
3. Using the formula catalogue, define length, width and thickness of the rectangle.
4. Using knowledge command fix the values
5. Fix relation to each parameter
6. Using formula editor command, change all the parameters accordingly and verify the model for altered dimensions.

Result: Hence, a 3D parametric model was developed by adjusting key parameters.



Experiment – 7 : Slice a 3D model and print a simple part using a 3d printer

Aim: To Slice a 3D model and print a simple part using a 3d printer.

Software required: Catia R21 V5, Ultimaker Cura.

Commands required: Sketcher, Part module (Catia), Prepare preview, print settings, slice.

Procedure:

Step-1: Build Cylindrical model of given dimensions in catia software using sketcher and part modules.

Step-2: Convert the cylindrical part (.cat part) into (.stl format).

Step-3: Import the given saved .stl file into Cura software.

Step-4: Prepare the model for proper print settings. infill density 40%, infill pattern 'grid', shell thickness 0.3mm, layer thickness 0.1mm.

Step-5: Slice the model and preview the preview the process.

Step-6: Save the file to disk and convert into G- code format.

Step-7: Load the G-code to 3D printer for preparing the cylinder.

Step-8: Pre-heat the bed and extruder for 70°C and 210°C.

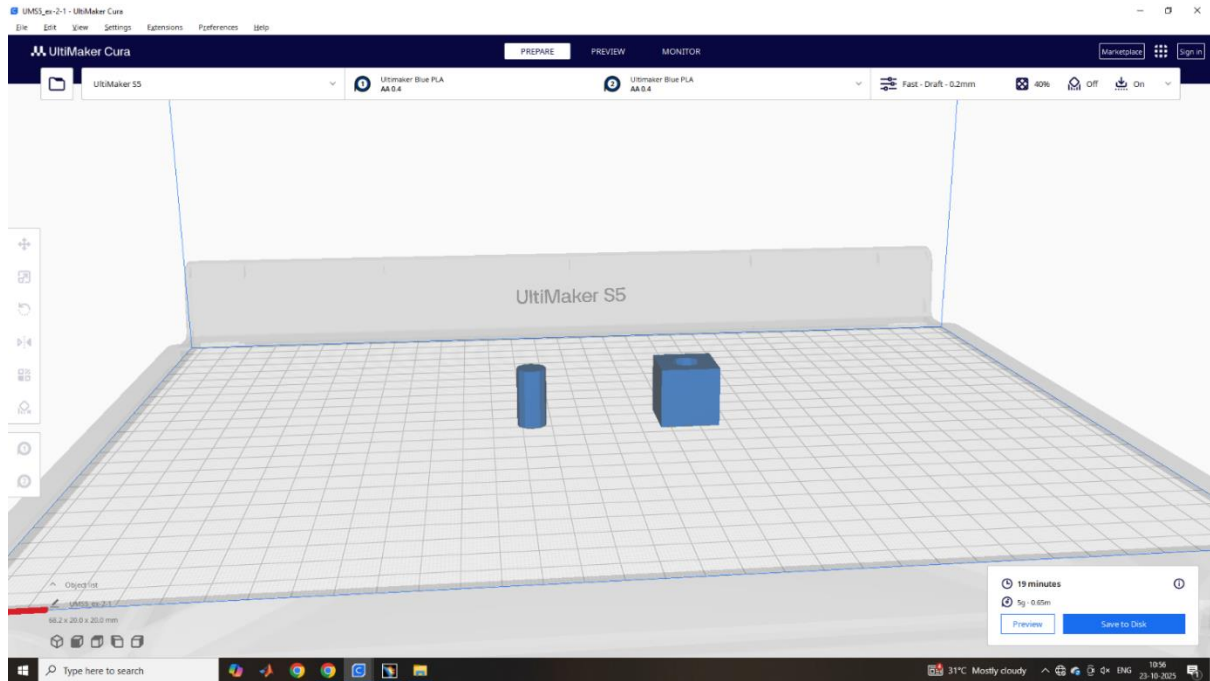
Step-9: Select the model and press start button.

G-Code:

```
M82 ;absolute extrusion mode
G92 E0
M190 S60
M104 S205
M109 S205
G280 S1
G0 Z20.001
G1 F2700 E-6.5
;LAYER_COUNT:100
G0 F9000 X330 Y237
;LAYER:0
M106 S255
M204 S1500
M205 X30 Y30
G1 F600 Z2.2
G0 F9000 X180.886 Y126.597 Z2.2
```

```
;TYPE:SKIRT
G1 F600 Z0.2
G1 F2700 E0
G1 F1800 X177.176 Y132.176 E0.08402
G1 X171.597 Y135.886 E0.16804
G1 X165.001 Y137.211 E0.25241
G1 X158.398 Y135.884 E0.33687
G1 X152.825 Y132.178 E0.4208
G1 X149.113 Y126.596 E0.50486
G1 X147.789 Y120 E0.58923
G1 X149.112 Y113.405 E0.67358
G1 X152.824 Y107.824 E0.75763
G1 X158.405 Y104.112 E0.84169
G1 X165 Y102.789 E0.92604
G1 X171.596 Y104.113 E1.01041
G1 X177.178 Y107.825 E1.09447
G1 X180.884 Y113.398 E1.1784
G1 X182.211 Y120.001 E1.26286
G1 X180.886 Y126.597 E1.34723
G0 F9000 X180.509 Y126.442
G1 F1800 X176.888 Y131.888 E1.42924
```

Result : The cylinder(PLA material) was prepared using 3D printing machine.



Experiment – 8 : Print and assemble multiple components,ensuring proper fit

Aim: To Print and assemble multiple components,ensuring proper fit.

Software required: Catia R21 V5,Ultimaker Cura.

Commands required: Sketcher,Part module (Catia), Prepare preview, print settings,slice.

Procedure:

Step-1: Build Cylindrical model of given dimensions in catia software using sketcher and part modules.

Step-2: Convert the cylindrical part (.cat part) into (.stl format).

Step-3: Import the given saved .stl file into Cura software.

Step-4: Prepare the model for proper print settings.infill density 40%,infill pattern 'grid',shell thickness 0.3mm, layer thickness 0.1mm.

Step-5: use support structure in the print settings as the given part model is inclined.

Step-6: Slice the model and preview the preview the process.

Step-7: Save the file to disk and convert into G- code format.

Step-8: Load the G-code to 3D printer for preparing the cylinder.

Step-9: Pre-heat the bed and extruder for 70°C and 210°C.

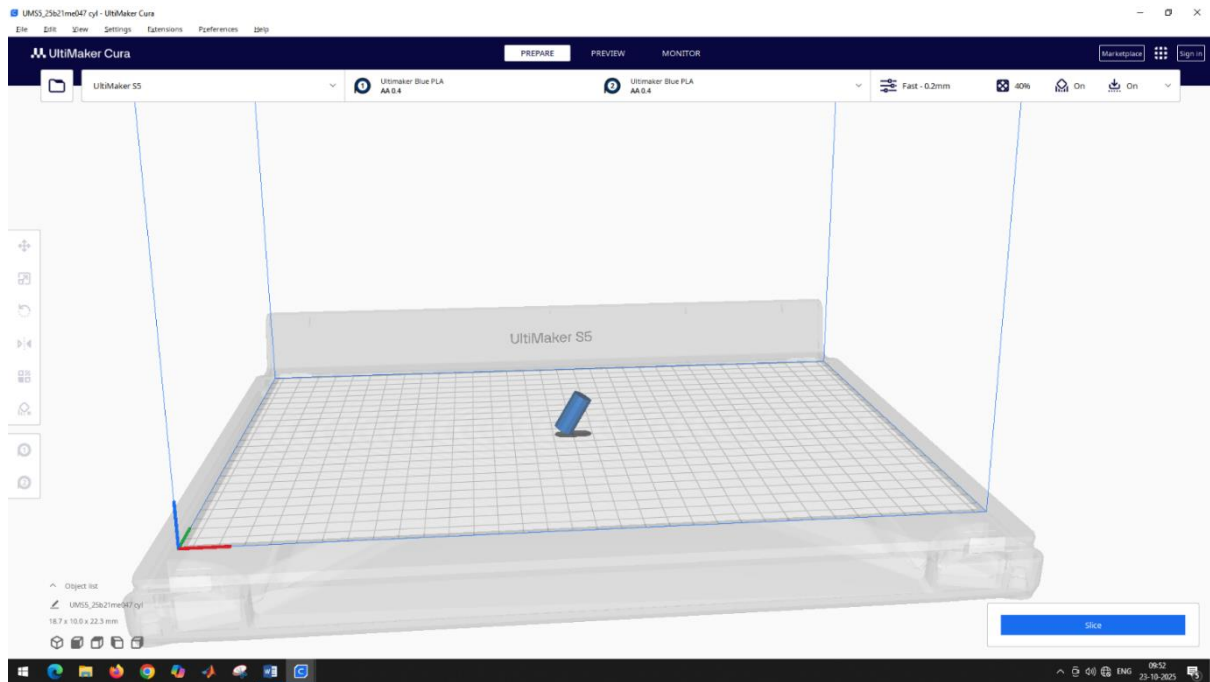
Step-10: Select the model and press start button.

G-Code:

```
M82 ;absolute extrusion mode
G92 E0
M190 S60
M104 S205
M109 S205
G280 S1
G0 Z20.001
G1 F2700 E-6.5
;LAYER_COUNT:100
G0 F9000 X330 Y237
;LAYER:0
M106 S255
M204 S2000
M205 X30 Y30
```

G1 F600 Z2.2
G0 F9000 X207.963 Y137.101 Z2.2
;TYPE:SKIRT
G1 F600 Z0.2
G1 F2700 E0
G1 F1800 X205.402 Y139.83 E0.04693
G1 X202.32 Y140.752 E0.08727
G1 X180.707 Y140.752 E0.35831
G1 X178.546 Y140.158 E0.38641
G1 X176.421 Y138.529 E0.41999
G1 X175.04 Y136.414 E0.45167
G1 X174.528 Y134.222 E0.4799
G1 X174.528 Y112.426 E0.75323
G1 X175.377 Y110.199 E0.78311
G1 X176.889 Y108.424 E0.81235
G1 X178.87 Y107.122 E0.84208
G1 X181.054 Y106.612 E0.87021
G1 X202.854 Y106.612 E1.14359
G1 X205.081 Y107.461 E1.17347
G1 X206.856 Y108.973 E1.20272
G1 X208.158 Y110.954 E1.23244
G1 X208.668 Y113.138 E1.26057
G1 X208.668 Y134.644 E1.53026
G1 X207.963 Y137.101 E1.56232
G0 F9000 X207.606 Y136.896
G1 F1800 X205.183 Y139.479 E1.60673

Result : Hence, multiple components were printed and assembled ensuring proper fit.



Experiment – 9 : Print complex geometries using support structures

Aim: To Print complex geometries using support structures.

Software required: Catia R21 V5, Ultimaker Cura.

Commands required: Sketcher, Part module (Catia), Prepare preview, print settings, slice.

Procedure:

Step-1: Build Cylindrical model of given dimensions in catia software using sketcher and part modules.

Step-2: Convert the cylindrical part (.cat part) into (.stl format).

Step-3: Import the given saved .stl file into Cura software.

Step-4: Prepare the model for proper print settings. infill density 40%, infill pattern 'grid', shell thickness 0.3mm, layer thickness 0.1mm.

Step-5: use support structure in the print settings as the given part model is inclined.

Step-6: Slice the model and preview the preview the process.

Step-7: Save the file to disk and convert into G- code format.

Step-8: Load the G-code to 3D printer for preparing the cylinder.

Step-9: Pre-heat the bed and extruder for 70°C and 210°C.

Step-10: Select the model and press start button.

G-code:

```
G1 F600 Z0.2
G1 F2700 E0
G1 F1800 X169.435 Y125.005 E0.02239
G1 X167.776 Y126.352 E0.04919
G1 X167.005 Y126.624 E0.05945
G1 X166.751 Y128.239 E0.07995
G1 X165.851 Y129.887 E0.1035
G1 X164.264 Y131.655 E0.13329
G1 X162.341 Y132.656 E0.16048
G1 X159.376 Y133.477 E0.19906
G1 X157.417 Y133.477 E0.22362
G1 X154.205 Y132.547 E0.26556
G1 X152.481 Y131.487 E0.29094
G1 X150.365 Y129.371 E0.32846
```

G1 X149.276 Y127.525 E0.35534
G1 X148.418 Y124.601 E0.39355
G1 X148.415 Y122.401 E0.42114
G1 X148.596 Y121.719 E0.42999
G1 X148.418 Y120.964 E0.43972
G1 X148.415 Y118.956 E0.4649
G1 X148.611 Y118.181 E0.47493
G1 X148.451 Y117.52 E0.48345
G1 X148.357 Y115.487 E0.50898
G1 X149.366 Y112.024 E0.55421
G1 X150.02 Y110.871 E0.57083
G1 X152.295 Y108.465 E0.61236
G1 X154.154 Y107.295 E0.6399
G1 X157.255 Y106.435 E0.68026
G1 X159.325 Y106.358 E0.70623
G1 X162.433 Y107.228 E0.74671
G1 X164.689 Y108.516 E0.77928
G1 X166.301 Y110.695 E0.81327
G1 X167.074 Y112.731 E0.84059
G1 X167.101 Y113.425 E0.84929
G1 X167.931 Y113.767 E0.86055
G1 X169.748 Y115.245 E0.88992
G1 X171.237 Y117.839 E0.92743
G1 X171.456 Y121.248 E0.97027
G1 X170.502 Y123.573 E1.00179
G0 F9000 X170.151 Y123.374
G1 F1800 X169.144 Y124.726 E1.02293

Result : The cylinder(PLA material) was prepared using support structures in 3D printing machine.

Experiment 10: Use Arduino and a Potentiometer to Control DC Motor Speed

Aim

To control the speed of a DC motor using an Arduino board and a potentiometer as an input device.

Apparatus/Requirements

- Arduino Uno board
- Breadboard and jumper wires
- 10k Ω potentiometer
- DC motor (5–12V)
- L293D motor driver IC or L298N motor driver module
- USB cable for Arduino
- Arduino IDE installed on computer

Theory

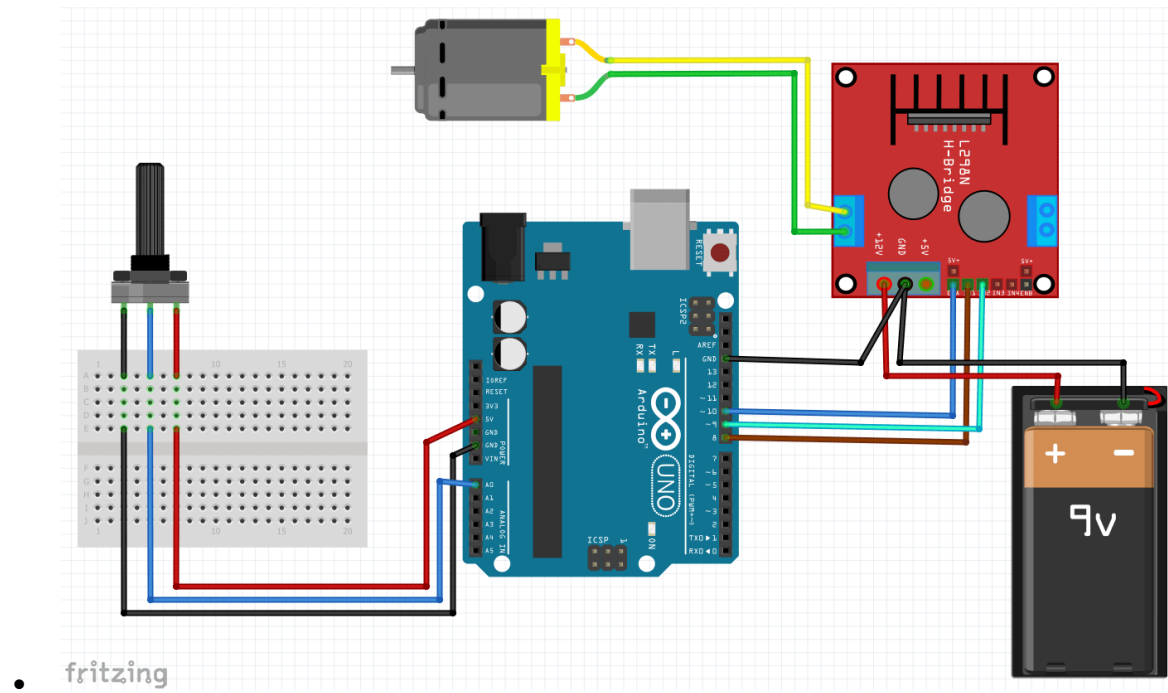
A **potentiometer** is a variable resistor that provides an analog voltage signal when its knob is rotated. Arduino reads this signal through its ADC (0–1023). This value is mapped into a PWM (Pulse Width Modulation) signal to vary the motor speed. The motor driver (L293D/L298N) acts as an interface because Arduino cannot supply enough current to drive a motor directly.

Concept:

- Potentiometer value $\rightarrow$ `analogRead()` (0–1023)
- Map to PWM $\rightarrow$ `analogWrite()` (0–255)
- PWM applied to motor driver input $\rightarrow$ motor speed changes

Circuit Description

- Potentiometer middle pin $\rightarrow$ A0 of Arduino
- Motor driver input $\rightarrow$ Arduino pin 10 (PWM)
- Motor driver output $\rightarrow$ DC motor terminals
- Vcc (5V) $\rightarrow$ Potentiometer and motor driver
- GND common



Procedure

1. Connect the potentiometer to A0 pin of Arduino.
2. Connect DC motor via L293D motor driver to Arduino pin 9.
3. Connect Arduino to PC and open Arduino IDE.
4. Upload the program that reads potentiometer value and generates PWM.
5. Rotate the potentiometer knob and observe motor speed change.

Sample Code

```
int potPin = A0;  
  
int in1=8;  
  
int in2=9;  
  
int motorPin = 10;  
  
int potValue = 0;  
  
int motorSpeed = 0;
```

```

void setup( )
{
  pinMode(motorPin, OUTPUT);
  pinMode(in1, OUTPUT);
  pinMode(in2, OUTPUT);
  digitalWrite(in1, HIGH);
  digitalWrite(in2, LOW);
}

void loop() {
  potValue = analogRead(potPin);
  motorSpeed = map(potValue, 0, 1023, 0, 255);
  analogWrite(motorPin, motorSpeed);
}

```

Observations

Potentiometer Position	ADC Value (0–1023)	PWM Value (0–255)	Motor Behaviour / RPM (qualitative)
Min (fully CCW)	~0	0	Motor stopped
Low	~200	~50	Slow rotation
Mid	~512	~128	Moderate speed
High	~800	~200	Fast
Max (fully CW)	~1023	255	Maximum speed (no load)

Precautions

1. Use common ground between Arduino, motor power supply and driver.
2. Never drive the motor directly from Arduino pins — always use the motor driver IC or module.
3. Add a flyback diode or use drivers with internal protection to avoid spikes.
4. Avoid touching spinning motor shaft; secure motor before testing.

5. Keep wiring short and firmly connected; avoid loose jumpers.
6. If you hear excessive noise or smell burning, immediately power off.

Result

The potentiometer input was read by the Arduino ADC and mapped to PWM output. Varying the potentiometer adjusted the PWM duty cycle and thus the motor speed. The system successfully demonstrated closed-loop-like control (human-in-the-loop) of motor speed using Arduino PWM.

Experiment 11 — Use Arduino to Control a Servo Motor's Angular Position

Aim

To control the angular position of a servo motor using an Arduino, and to demonstrate precise angular positioning using PWM signals.

Apparatus / Requirements

- Arduino Uno (or equivalent)
- Hobby servo motor (e.g., SG90 or MG90S)
- Breadboard and jumper wires
- External 5V supply (recommended for metal/torque servos) or Arduino 5V for small servos
- USB cable and PC with Arduino IDE

Theory / Background

A hobby servo contains a DC motor, gearbox, potentiometer (position feedback), and a control circuit. The servo expects a periodic control pulse (typically 20 ms period) where pulse width (usually ~1 ms to ~2 ms) determines shaft angle (0° to ~180°). Arduino's Servo library generates appropriate pulses without manual PWM timing.

Circuit Description (text)

- Servo red (Vcc) → +5V (use external 5V if servo draws >500 mA)
- Servo brown/black (GND) → GND (Arduino and external supply common ground)
- Servo yellow/white (control) → Arduino digital pin 9 (or any PWM/digital pin used by Servo library)

Note: If using external power for servo, connect grounds together (Arduino GND ↔ external supply GND).

Procedure

1. Mount the servo to the bench or test rig with its horn attached.
2. Wire power, ground and control pin as described. If using external power, confirm +5V and GND polarity.
3. Open Arduino IDE and create a new sketch. Include `#include <Servo.h>`.
4. Upload the sketch that sweeps the servo or sets it to specific angles.
5. Observe the servo movement and note accuracy, jitter, and any missed positions.
6. Test moving the servo to fixed angles (0°, 45°, 90°, 135°, 180°) and record behaviour.

Sample Code — Sweep and Position

```
#include <Servo.h>

Servo myServo; // create servo object
int servoPin = 9;

void setup() {

void loop() {
    // Sweep from 0 to 180 and back
    for (int pos = 0; pos <= 180; pos += 1) {
        myServo.write(pos);
        delay(10); // small delay for smooth motion
    }
    delay(500);
    for (int pos = 180; pos >= 0; pos -= 1) {
myServo.attach(servoPin); // attach to pin 9
        delay(500);
    }

    // Hold at specific positions (optional)
    myServo.write(90); // center
    delay(1000);
    myServo.write(45);
    delay(1000);
}
```



Observations (example)

Commanded Angle (°)	Observed Angle (qualitative)	Accuracy/Remarks
0	Close to 0°	Good; small jitter
45	~45°	Accurate
90	~90°	Centered
180	~180°	At end stop; be careful of torque

Precautions

1. Use an appropriate power source: metal servos draw higher current — never power from Arduino 5V pin if current is high.
2. Always connect common ground between supplies.
3. Avoid stalling the servo for long periods at high torque; it can overheat.
4. Do not apply excessive external force to the shaft while powered.
5. Mount servo securely to prevent sudden movements from causing injury.

Result

The Arduino successfully controlled the servo's angular position using the Servo library. The servo responded to commanded angles smoothly and repeatably. The experiment demonstrates position control via pulse-width based commands.

Experiment 12 — Build and Program an Arduino Robot to Avoid Obstacles Using Ultrasonic Sensors

Aim

To design, assemble and program a two-wheeled Arduino-based robot that uses an ultrasonic sensor to detect obstacles and autonomously avoid collisions.

Apparatus / Requirements

- Arduino Uno (or Nano)
- Chassis with two DC motors + wheels (differential drive)
- Motor driver module (L298N, L293D or similar)
- HC-SR04 ultrasonic distance sensor (or equivalent)
- Caster wheel (for balance)
- Battery pack (e.g., 7.4V Li-ion or 6×AA pack depending on motors)
- Jumper wires, breadboard, screws, mounting hardware
- Optional: LEDs, buzzer for debug signals

Theory / Background

The HC-SR04 ultrasonic sensor measures distance by emitting an ultrasonic pulse (TRIG) and measuring the echo return time (ECHO). $\text{Distance} = (\text{time} \times \text{speed_of_sound})/2$. The robot uses distance thresholds to decide motion: move forward when path clear, stop and turn (left/right) when an obstacle is detected within threshold.

Basic behaviour (simple reactive algorithm):

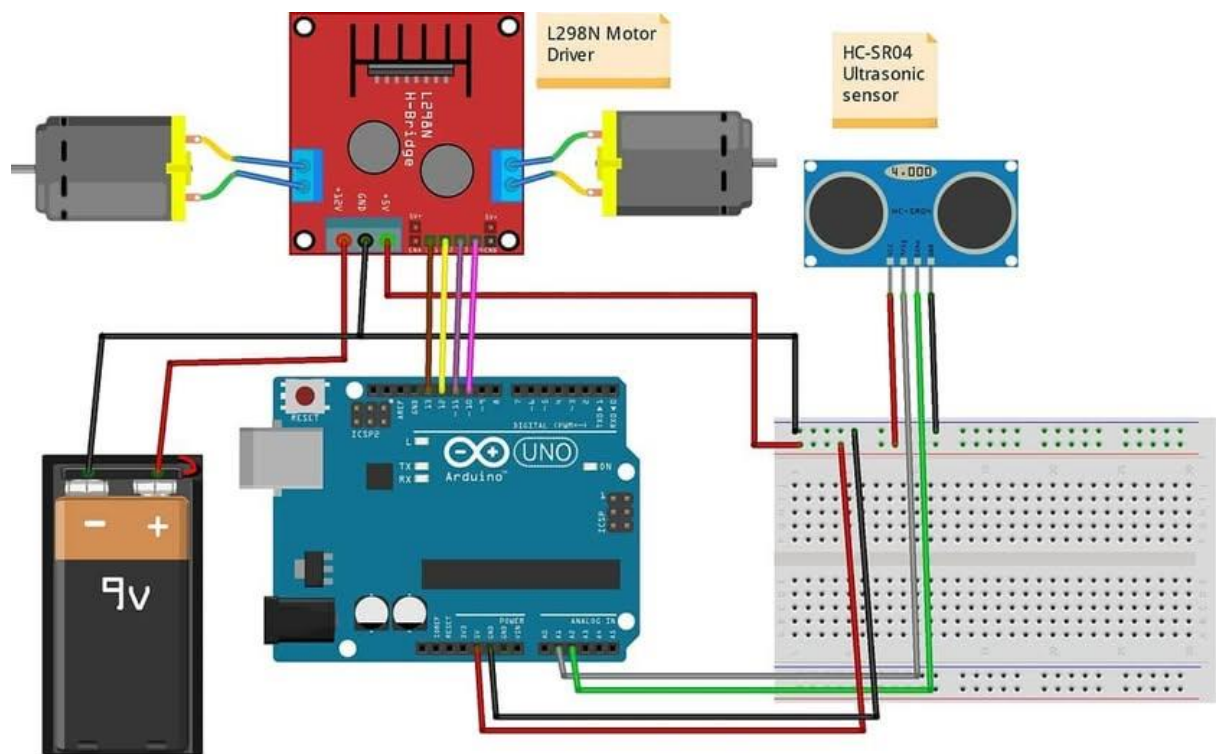
- If distance > safe_distance: move forward.
- If distance ≤ safe_distance: stop, back up slightly, turn (choose direction), then resume.

Circuit Description

- HC-SR04:
 - Vcc → +5V
 - GND → GND
 - TRIG → Arduino digital pin 10
 - ECHO → Arduino digital pin 11
- Motors:
 - Motor A terminals → Motor driver OUT1, OUT2

- Motor B terminals → Motor driver OUT3, OUT4
- Motor driver IN pins → Arduino control pins (for example IN1=2, IN2=3, IN3=4, IN4=5)
- Motor driver EN pins (enable PWM) → Arduino PWM pins (e.g., ENA=9, ENB=10) for speed control (optional)
- Power:
 - Motor power supply (battery) → motor driver Vcc (Vmotor)
 - Logic 5V → Arduino 5V (if motor driver has regulator, ensure no conflicting power sources)
 - Common ground between Arduino and motor supply.

Ensure correct voltage limits for HC-SR04 (5V).



fritzing

Flowchart / Algorithm

1. Initialize pins and sensors.
2. Loop:
 - Read distance from ultrasonic sensor.
 - If distance > threshold (e.g., 20–25 cm): move forward.
 - Else (obstacle close): stop, back up for ~300 ms, then turn (e.g., right) for ~400 ms, then continue.
3. Repeat.

Procedure

1. Assemble chassis: mount motors, wheels and caster. Mount motor driver and Arduino on chassis.
2. Wire motors to motor driver and motor driver to Arduino as per circuit description.
3. Mount ultrasonic sensor at front (facing forward) and wire it to Arduino.
4. Connect battery pack to motor driver (observing polarity). Connect Arduino to its power source (USB for programming; later power from battery or regulated 5V). Ensure common ground.
5. Open Arduino IDE, write/ upload the obstacle avoidance sketch.
6. Place robot on a flat surface and run. Observe reactions to obstacles (box, hand, wall).
7. Tune threshold distances and turn timings for reliable avoidance.

Sample Code (basic)

// Pins

const int trigPin = 10;

const int echoPin = 11;

// Motor control pins

const int in1 = 2;

const int in2 = 3;

const int in3 = 4;

const int in4 = 5;

```
const int ena = 9; // PWM for left motor speed
const int enb = 6; // PWM for right motor speed

long duration;
int distance;
int threshold = 25; // centimeters

void setup() {
  pinMode(trigPin, OUTPUT);
  pinMode(echoPin, INPUT);
  pinMode(in1, OUTPUT);
  pinMode(in2, OUTPUT);
  pinMode(in3, OUTPUT);
  pinMode(in4, OUTPUT);
  pinMode(ena, OUTPUT);
  pinMode(enb, OUTPUT);
  Serial.begin(9600);
}

int readDistance() {
  digitalWrite(trigPin, LOW);
  delayMicroseconds(2);
  digitalWrite(trigPin, HIGH);
  delayMicroseconds(10);
  digitalWrite(trigPin, LOW);
  duration = pulseIn(echoPin, HIGH, 30000); // timeout 30ms
  distance = duration * 0.034 / 2; // cm
  if (duration == 0) distance = 500; // no echo
  return distance;
}
```

```
}
```

```
void forward(int speed) {  
    analogWrite(ena, speed);  
    analogWrite(enb, speed);  
    digitalWrite(in1, HIGH);  
    digitalWrite(in2, LOW);  
    digitalWrite(in3, HIGH);  
    digitalWrite(in4, LOW);  
}
```

```
void back(int speed) {  
    analogWrite(ena, speed);  
    analogWrite(enb, speed);  
    digitalWrite(in1, LOW);  
    digitalWrite(in2, HIGH);  
    digitalWrite(in3, LOW);  
    digitalWrite(in4, HIGH);  
}
```

```
void stopMotors() {  
    digitalWrite(in1, LOW); digitalWrite(in2, LOW);  
    digitalWrite(in3, LOW); digitalWrite(in4, LOW);  
    analogWrite(ena, 0); analogWrite(enb, 0);  
}
```

```
void setupTurnRight(int speed) {  
    analogWrite(ena, speed);  
    analogWrite(enb, speed);
```

```
// left forward, right backward -> rotate right  
digitalWrite(in1, HIGH);  
digitalWrite(in2, LOW);  
digitalWrite(in3, LOW);  
digitalWrite(in4, HIGH);  
}
```

```
void loop() {  
  int d = readDistance();  
  Serial.print("Distance: "); Serial.println(d);  
  if (d > threshold) {  
    forward(180); // move forward  
  } else {  
    stopMotors();  
    delay(100);  
    back(180);  
    delay(300); // back a bit  
    stopMotors();  
    delay(100);  
    // turn right  
    setupTurnRight(180);  
    delay(400);  
    stopMotors();  
    delay(100);  
  }  
}
```

Observations (example)

Trial	Obstacle Distance (cm)	Robot Response	Success/Fail	Notes
1	50	Move forward	Success	Smooth
2	20	Stop → Back → Turn → Forward	Success	Minor hesitation
3	10	Quick reversal and turn	Success	Good avoidance
4	5	Sometimes no detection (too close)	Fail	Reduce threshold or improve timing

Tuning Tips

- Change threshold to tune for sensor/robot geometry.
- Increase back/turn durations for longer or heavier robots.
- Add side ultrasonic sensors (left/right) for smarter turning decisions.
- Implement scanning (rotate sensor) or mapping for advanced avoidance.

Precautions

1. Ensure motor driver and Arduino grounds are common.
2. Observe correct supply voltages for motors and Arduino; do not feed >5V to Arduino pins.
3. Use proper current-rated wires and battery connectors.
4. Test motors at low speed first to verify wiring direction.
5. Watch for overheating of driver IC; provide proper heat sinking if needed.
6. Avoid operating near fragile objects until robot behaviour is verified.

Result

A simple reactive obstacle-avoidance robot was built and programmed. The ultrasonic sensor provided distance data that the Arduino used to detect obstacles and trigger avoidance manoeuvres (back up + turn). The robot was able to navigate simple obstacle courses reliably after tuning threshold and timings.